CS510 Project Report: State-aware Notebook-Based Coding Agent for AI and Data Science

Billy Li
zl20@illinois.edu

Haocheng Xia
hxia7@illinois.edu

Fulin Wang
fulinw2@illinois.edu

Abstract

Notebooks (e.g., Jupyter) are commonly used for handling diverse operations of AI and Data Science , within which there is a recent growth in popularity in using coding agents to compose end-to-end AI and data science pipelines, i.e., 'AutoKaggle'. These agents mimic humans, iteratively performing steps such as data cleaning, model training, and visualization. However, existing agent workflows contain major missed opportunities and inefficiencies: this is because unlike prior workflows performed by humans, existing agents crucially lack an understanding of mid-workflow states, being only able to observe (1) code in prior cells, and (2) end-to-end results (e.g., the score on the 'submission.csv'), which can result in mid-workflow issues being compounded upon, resulting in poor performance.

In this project, we build a new coding agent, called AI/DS-Agent, to bridge this gap between existing agent-based frameworks and human behavior. AI/DS-Agent's architecture induces agent awareness of both the notebook state and the state of the workflow via three key mechanisms: First, the *confidence module* instructs the agent to inspect state data when confidence for correctly composing the next cell is low; next, the *memory module* holds summarized error messages from prior failed execution attempts to facilitate more efficient debugging; finally, the *review module* promts the agent to repeat certain trajectories, selecting the final output to return to the user via majority voting. We show that AI/DS-Agent outperforms existing state-of-the-art frameworks (e.g., DatawiseAgent) in terms of task accuracy on the widely-used *Infiagent-DABench* dataset.

1 INTRODUCTION

Computational notebooks (e.g., Jupyter [20, 43], Rstudio [36]) are widely used by data scientists [34, 35]. A key feature of the notebook workflow is iterative code execution and result observation [1, 4], which is highly compatible with the incremental nature of data science tasks, such as interactive tutorials [19], data exploration [9, 10, 52], visualization [12], and model tuning [3, 44]. Recently, motivated by two key improvements made to Large Language Models (LLMs)—① the ability to invoke and use tools (i.e., *LLM Agents*) [14, 53] and ② the ability to effectively perform multi-turn interactions [8, 47], works have started studying the building of *notebook-centric AI and data science agents* [25, 27, 37, 51]: they aim to automate iterative interactions notebooks and achieve significant speedups in code generation [33], result understanding (thus reducing *think time* [12] between operations), or even run parallel sessions [32] to perform AI and data science (AI/DS) workflows at scale.

Problem: Gap between Human and Agent Behavior. While existing AI/DS notebook agents correctly capture the behavior of human practicioners, i.e., following the sequential loop of writing and running cell code, then observing output, they fail to mimic *how* humans peform each individual step: First, for observing output, humans often use mechanisms such as the variable inspector or commands such as `df.head()` to inspect results in detail [40]; yet, coding agents essentially operate blind to state data, being able to only observe code in prior cells; this can result in errors such as accessing non-existant fields in variables [25], or worse, *silent errors* that get compounded upon during workflows [13]. Second, for code generation, while humans often explicitly track prior errors during debugging [41], existing agent frameworks track errors implicitly, i.e., not distinguishing error messages and normal cell outputs [25, 51]. This can result in prior encountered errors being lost in long workflows [26], often resulting in agents repeatedly re-generating code with the same mistakes. Finally, human users often try multiple approaches for each important step (e.g., model selection) [48]; however, existing frameworks treat successful submission (i.e., instead of a *good* submission) as a goal [49], and consider the task as finished as soon as the first valid end-to-end result (i.e., the 'submission.csv').

Bridging the Human-Agent Gap. Given these observations, we hypothesize that aligning the behavior of AI/DS agent frameworks and human practicioners can bring significant performance benefits for the former. For example, recent works [37] have demonstrated that even simple mechanisms for outputting state data (variable names and types) can increase submission success rate and decrease end-to-end token usage; yet, human users often perform significantly more complex inspections such as on info of individual variables (e.g., `df.shape`) or even what fields are present in a variable (i.e., `dir(var)`) [21], hence, we expect higher benefits are possible with a more comprehensive approach. Also, mechanisms for explicitly tracking errors [50] and self-improvement [46] have seen successes in the adjacent applications of computer use and visual understanding, respectively.

Our Proposal. In this project, we propose AI/DS-Agent, a notebook and workflow state-aware agent framework for notebook-based AI/DS workflows. AI/DS-Agent is integrated with JupyterLab: given a user task description, it interacts with a Jupyter notebook instance by iterating between sending code executions to the notebook session and reading the code execution output. AI/DS-Agent utilizes 3 modules to address our observed gaps: first, for inspecting state data, the *confidence module* takes inspiration from techniques used in NL2SQL [6] and tracks the agent's confidence level in correctly generating cell code, then, provides a comprehensive selection of code executions for inspecting state data should confidence be low. Second, for error-aware code generation, the *memory module* keeps track of recent encountered errors, and associated failed debugging attempts, which it combines with explicit prompting to guide the agent to avoid past mistakes. Finally, observing recent successes in best-of-K methods for generating high-quality responses [11], the *review module* prompts the agent to explore

Table 1: Comparison between AI/DS-Agent **and other tools for enhancing notebook-based AI and Data Science workflows**

Approach	Mechanism
Cell execution recommendation [23, 24, 28]	Suggests cells for specific AI/DS subtasks (does not generalize to end-to-end workflows)
Data understanding [2, 30, 31]	Enables LLMs to understand specific datatypes (but not *when* to invoke understanding)
Agent-based AI/DS frameworks [22, 25, 27, 32, 37, 51]	Automates notebook-based AI/DS workflow composition (but deviates from human behavior)
Ours (AI/DS-Agent**)**	**Performant agent-based AI/DS frameworks that matches human behavior**

multiple code executions for important steps which are then aggregated via majority voting. Additionally, to avoid the verification-generation gap [38], AI/DS-Agent contains a lightweight filtering mechanism that removes errorneous results from voting.

Difference from Existing Work. There are existing works for enhancing AI/DS workflows in notebooks, compared to which AI/DS-Agent is significantly different. For example, prior to LLMs, there exists many works for recommending next cell executions to run [23, 24] (and what not to run [28]); however, these works target specific sub-tasks in AI/DS and utilize appropriately specific techniques (e.g., lightweight RNN [24], dataframe inspection [23], and lineage tracing [28]), which is orthogonal to the end-to-end workflows that AI/DS-Agent target. There also exists many works for understanding *specific* state data and code outputs with LLMs such as tables [2], charts [30], and JSON structures [31]; AI/DS-Agent is complimentary with these works and studies how to instruct LLMs on *when* to inspect and understand state data. Finally, There exists many agent-based AI/DS frameworks that are capable of composing end-to-end workflows [22, 25, 27, 32, 37, 51]; these works primarily focus on how to structure the agent-based execution loop (e.g., specialized agents for each subtask [27]) and/or architectural improvements (e.g., parallel sessions [32]), while AI/DS-Agent uniquely studies *what* to pass as inputs to agents and what to execute to better mimic human usage. table 1 summarizes differences.

Report structure. The report will be structured as follows:

- **Related Work.** We overview works related to our system. (§2)
- **System Overview.** We describe our system architecture and how it performs agent-based AI and data science workflows. (§3)
- **Modules.** We cover the confidence, memory, and review modules in detail and discuss how their inclusions enhance the performance of agent-based AI/DS frameworks. (§4)
- **Evaluation.** We show that AI/DS-Agent increases question success rate on the widely-used Infiagent-DABench dataset. (§5)

2 RELATED WORK

In this section, we discuss related work in notebook-based AI/DS framework architecture, usage of agent memory, and self-verification techniques for LLM agents.

Notebook-based AI/DS Framework Architecture. There exists a variety of works studying the architecture for notebook-based AI/DS agent frameworks [22, 25, 27, 32, 37, 51]. Earlier works focus on the general architecture: AutoKaggle [25] proposes a multi-agent framework with each responsible for an AI/DS subtask, while ML-master [27] and KompeteAI [22] study effective containerization of workflows; however, all of these works run standalone scripts rather than use a persistent notebook session, leading to inefficiencies from operations (e.g., data loading) being repeated between iterations. Follow-up work such as DatawiseAgent [51] address this issue by interacting with notebook sessions, utilizing new, reliable methods for communication (e.g., Jupyter MCP [15]). CaveAgent [37] further studies how to effectively pass notebook cell outputs back to the agent; DSGym [32] runs multiple agent-notebook trajectories in parallel. In comparison, AI/DS-Agent effectively improves on the rule-based information passing (i.e., variable names and types) in CaveAgent, while adopting established techniques (i.e., agent memory, self-verification) used in adjacent problem settings for more performance gains.

Agent Memory. Agent memory is commonly used to make multi-step agents less myopic by carrying useful information from prior interactions into later decisions. In code-generation and tool-use settings, this memory can include previous errors, successful tool outputs, summaries of intermediate state, or self-reflection traces [39, 50]. Such mechanisms are especially relevant for notebook agents because execution state is continuously distributed across many cells, and important evidence can be buried among long stdout outputs, stack traces, and natural-language explanations, which aligns with the demands of long-horizon execution. However, prior work also suggests a tension: adding more memory increases the chance that useful evidence remains available, but it also increases context length and may distract smaller models from the current instruction [26]. Our memory module is designed to accommodate this trade-off. Rather than treating memory as an always-beneficial store of all prior information, we study what information should be preserved, compressed, or verified at the final-answer level.

Self-Consistency and Self-Verification for LLM Agents. A recurring problem with LLM agents is that each single run can generate unexpected results by sampling noise, schema mistakes, or premature termination, even when the agent is in principle capable of solving the task. There are two lines of work studing mitigation techniques that do not involve retraining the underlying model: ① Aggregate multiple sampled trajectories, where Self-Consistency [45] shows that sampling several chain-of-thought completions and taking the majority answer substantially improves arithmetic and commonsense reasoning, which is later generalized to free-form outputs by Universal Self-Consistency [7]. For coding and data analysis, similar voting strategies have been used to aggregate executed programs such as AlphaCode-style filtering and CodeT [5], which uses generated test cases as a voting signal. ② Asking the model to audit its own output, where Self-Refine [29] and Reflexion

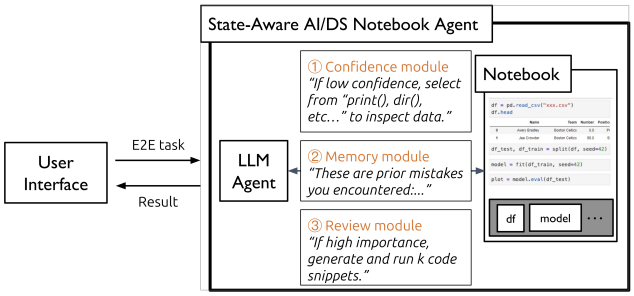

Figure 1: AI/DS-Agent architecture. AI/DS-Agent acts as a gateway between the user interface and the LLMs; it intercepts queries to perform optimizations and achieve higher LLM throughput.

[39] use generated critiques to trigger further revision, and CRITIC [16] grounds the critique with external tools. However, Huang et al. [18] and Valmeekam et al. [42] report that LLMs frequently fail to self-correct when given only their own feedback, suggesting that self-review is more reliable as a detector of failure than as a corrector. In comparison, AI/DS-Agent aims to combine these two lines of work for of these two lines and is specifically targeted at notebook-based AI/DS agents: Motivated by the underperformance of self-correction, our review module (§4.3) uses self-review only where evidence suggests it is reliable, and rely on self-consistency for the actual answer selection.

3 SYSTEM OVERVIEW

This section overviews AI/DS-Agent at a high level in terms of its components (§3.1) and workflow (§3.2).

3.1 AI/DS-Agent Components

AI/DS-Agent is an agent-based AI/DS framework; implementation-wise, it off the open-source DatawiseAgent [51]. Figure 1 depicts AI/DS-Agent's components.

Notebook session. AI/DS-Agent hosts a Jupyter notebook session for the agent to interact with on the user's behalf to complete the specified task. The agent interacts with the notebook session via the REST API: at each step, it ① sends the composed cell execution to the notebook session for execution, then ② processes the cell execution output returned from the notebook session.

LLM Agent. The LLM agent interacts with the notebook session and other modules in AI/DS-Agent to iteratively compose cell executions during a workflow. Implementation-wise, We use the widely-adopted GPT4o-mini in AI/DS-Agent.

Confidence Module. The confidence module is a prompting-based mechanism that instructs the LLM agent to output a score denoting its confidence in correctly generating the next cell execution. If this score is low, it instructs the LLM agent to inspect the variables in the notebook state via a separate cell execution, mimicking the behavior of human analysts (§4.1).

Memory Module. The memory module acts as a bolt-on component of the LLM agent, explicitly storing the error outputs of prior failed cell executions. These outputs are provided as inputs to the LLM agent when composing next cell executions to aid it in avoiding prior mistakes (§4.2).

Review Module. The review module is another prompting-based mechanism that instructs the LLM agent to generate and execute k instead of 1 cells for important step. Then, it evaluates the correctness of these outputs, and performs majority voting with the correct outputs to formulate the final output of the step (§4.3)

3.2 AI/DS-Agent Workflow

This section describes AI/DS-Agent's operations. Users issue end-to-end task descriptions (e.g., Kaggle-style competitions) to AI/DS-Agent, then, for each task, AI/DS-Agent initializes a Jupyter notebook session with which an agent interactively runs cell executions with to complete the given task.

Composing Cell Code. For each step, AI/DS-Agent composes and executes its corresponding cell execution as follows:

- The confidence module is invoked to determine the confidence of the agent in correctly generating the cell code.
- If confidence is low, the agent is instructed to choose from specialized *inspection executions* (e.g., `%who`, `print`, `dir(var)`) to run on the notebook session state. This process is repeated until confidence is above threshold, or if a maximum number of inspection rounds per step is reached.
- The review module is invoked to determine the importance of the current step. If the importance is above a threshold, the agent is instructed to sequentially write and execute k cell executions for the current step instead of 1.
- The memory module is invoked to provide prior error messages and failed cell executions for generating code cell(s).

The code cells are passed to the notebook session for execution.

Handling Cell Executions. After code cells are passed to the notebook session, AI/DS-Agent processes the results as follows:

- If an error is encountered, the error message and the current cell execution is appended to the memory module, and AI/DS-Agent restarts the code generation loop for the current step.
- Otherwise, the returned cell output is passed to the LLM agent to start the next step, unless the current task has been completed (e.g., a 'submission.csv' has been generated).

To avoid network-related issues, AI/DS-Agent sets a timeout of 10 minutes for each cell execution, upon which the cell execution is treated as errorneous and the code generation loop is restarted; we empirically verify this to not affect experiment task execution, as only small datasets are used (§5).

4 CACHING IN LLM-UPSCALER

This section describes the three core modules of AI/DS-Agent: the confidence module (§4.1), the memory module (§4.2), and the review module (§4.3).

4.1 Confidence Module

The Confidence module instructs the LLM agent to write read-only cell code for inspecting variables in the notebook session

State Inspection Prompt

Your confidence score is below the required threshold. To better understand the current workspace state before writing the next code block, generate one or more inspection code cells using appropriate IPython/pandas commands, for example:

- `%whos` or `%who_ls <type>` to list all variables with their types and sizes
- `print(variable_name)` to inspect a specific variable's value
- `df.info()`, `df.head()`, `df.describe()` for DataFrame inspection
- `type(variable)`, `len(variable)`, `dir(variable)` for generic object inspection

===assistant===
<confidence>0.9</confidence>
I am **confident** in proceeding to calculate the duration …
```
```python
# Inspecting the loaded DataFrame to understand its structure and data types
budget_data.info()
```
===user===
code execution
===assistant===
```markdown
<confidence>0.95</confidence>
I am **highly confident** in proceeding to calculate the duration…
```

Figure 2: Prompt of AI/DS-Agent's confidence module (top) and effect of state inspection (bottom). The agent gains confidence in its ability to correctly write and execute cell code given the `info()` function call.

state. After observing the outputs of each cell execution and prior to generating the code for the next task, the confidence module prompts the agent to score itself, proportion-wise from 0 to 1, on how confident it is to generate the next cell execution error-free. Then, if the score is under a threshold (we empirically study the effect of different thresholds in §5.2), the next cell generation is paused, and an intermediate state inspection is conducted: the agent is instructed to execute a read-only cell execution on the notebook state, with suggestions given for functions commonly used by human practicioners (prompt depicted in fig. 2, top). Then, after observing the variable data, the agent is instructed to re-evaluate its confidence (fig. 2, botom); the inspection process is repeated if the confidence is still below the threshold; otherwise, the agent continues to cell code generation via the memory (§4.2 and review (§4.3) modules.

To avoid the agent from being stuck in state inspection, AI/DS-Agent features a predetermined maximum rounds (we empirically set to 3) of state inspection the agent can perform per code cell generation. If the maximum is reached, the agent will proceed to cell generation even if the confidence is still lower than the threshold.

4.2 Memory Module and Answer Verification

Setup. On top of the DatawiseAgent-style notebook execution loop, we add a memory module that controls which intermediate execution evidence is carried across notebook steps. Given a user query q and a notebook task state S_i at step i, the original agent composes the next cell from the prompt context

$$x_i = \text{Serialize}(q, h_{<i}),$$

where $h_{<i}$ denotes the previous cells and their observed outputs. This unstructured serialization treats successful outputs, error traces, variable states, and final-answer constraints as the same kind of context. As a result, the agent is vulnerable to two opposite failure modes. If too much history is retained, long stdout outputs and irrelevant intermediate results can crowd out the current instruction. If too much history is trimmed, the agent can lose important question constraints or the required final-answer schema.

We therefore define memory as a controlled evidence store rather than as a raw transcript. At step i, the memory state is

$$M_i = (E_i, O_i, Z_i, F),$$

where E_i stores recent error evidence, O_i stores compressed successful-output evidence, Z_i stores lightweight variable-state summaries, and F stores final-answer format constraints extracted from the user query. The agent prompt is then constructed as

$$x_i = \text{Prompt}(q, g_i, \text{Code}_{<i}, M_i),$$

where g_i is the current step goal. This formulation makes memory an explicit part of the control policy rather than an accidental side effect of prompt serialization.

Memory Update Policy. After each notebook execution, the memory module updates M_i according to the observed result. If the execution fails, the corresponding cell and error trace are appended to E_i. If the execution succeeds, the raw output is compressed before being stored in O_i, while a lightweight snapshot of live variables may be stored in Z_i. Formally, for an executed cell u_i with observation y_i, we apply

$$M_{i+1} = \begin{cases} \text{UpdateError}(M_i, u_i, y_i), & y_i \in \mathcal{Y}_{\text{err}}, \\ \text{UpdateSuccess}(M_i, u_i, y_i, S_{i+1}), & y_i \in \mathcal{Y}_{\text{ok}}. \end{cases}$$

The error update preserves more detail than the success update because stack traces and failed code are often directly useful for repair. By contrast, successful outputs are truncated or summarized when they are long, since retaining full outputs can distract the model from the next step.

We implement four memory-related mechanisms. First, *debug-context trimming* constructs a shorter repair prompt by retaining the user query, the current step goal, condensed prior code, and the cells that caused the current error. Second, *successful-output truncation* shortens long successful outputs while preserving full error traces. Third, *variable snapshots* inject concise summaries of live variables, including names, types, and shapes. Fourth, *debug-error memory* summarizes prior failed debugging attempts and prepends them to the next debugging prompt, so the agent is less likely to repeat the same repair.
```

Answer Verification. The memory ablation in Section 5.3 shows that better memory alone is not sufficient for the small-model setting. In several cases, the agent computes the correct value but fails to emit the required final-answer format. This motivates a separate answer verification stage. Let $a = \text{Final}(\tau)$ be the final-answer block emitted by a trajectory τ. The verifier W checks whether a contains all required fields and whether they match the expected schema induced by F:

$$W(a, F) \in \{\textsc{Valid}, \textsc{Invalid}\}.$$

Unlike the review module in Section 4.3, this verifier is not a general reasoning judge. It does not rerun code, repair computation, or decide whether the analysis is statistically correct. Instead, it is a lightweight schema guardrail that checks whether the final response is extractable by the evaluator.

When $W(a, F) = \textsc{Invalid}$, the system invokes an extraction-only formatter:

$$a' = \text{Format}(a, F),$$

which rewrites the final answer into the required canonical form while preserving the computed values. For InfiAgent-Bench-style tasks, this primarily means recovering missing `@name[value]` fields from the agent's final explanation. For example, if the response contains the correct value in prose but omits the required token, the formatter emits the corresponding structured answer. The formatter is constrained not to introduce new computation; it can only extract and normalize values that already appear in the trajectory.

Final Design. Based on the case study, our final memory module is deliberately lightweight. It preserves high-value evidence such as recent error traces, failed repair attempts, concise successful-output summaries, and small variable snapshots, but avoids injecting large unstructured execution histories. We separate memory from answer verification: memory helps the agent decide what to do next during execution, while the verifier and canonical formatter ensure that the final answer is extractable and schema-compliant.

This separation is important for small notebook agents. More memory can increase the chance that useful evidence remains available, but it also increases prompt length and can distract the model from the current instruction. In contrast, final-answer verification targets the failure mode that appears most frequently in our ablation: the computation is often correct, but the answer is not emitted in the required form. Thus, AI/DS-Agent uses memory as a compact execution-evidence store and uses answer verification as a separate guardrail for final response compliance.

4.3 Review Module: Trajectory Auditing and Aggregation

Setup. The review module conducts the trajectory auditing and aggregation steps following methodology in DatawiseAgent [51]. With input q, T, the orginal DatawiseAgent produces a single trajectory $\tau \sim p_{\mathcal{A}}(\cdot \mid q, T, t)$ at temperature t through FST-orchestrated planning, execution, debugging, and filtering (Note, $\mathcal{A}$ is the original Datawiseagent). In our system, we write $\text{Final}(\tau)$ for its final-answer block and alleviate the unstableness of solving the problem only with a single τ. Our work adds a reviewer R, a signature-based aggregator V, a review-aware aggregator V_R as their combination, and per-trajectory runtime protection.

The Review Module. DatawiseAgent's post-filtering stage q_{filter} only classifies per-error debug outcomes mid-execution. Instead, our reviewer module runs only after the agent emits `<Fulfill USER INSTRUCTION>` as a single non-corrective audit. To be more specific, we add a new state q_{review} after the Datawiseagent pipeline, and define R with output v equal to `No_Issues` or `Issues_Found`. To avoid hallucination, R is designed with only judegment function and cannot emit repair code, re-execute, or return to q_{inc} or q_{debug}, which helps to seperate the LLM-as-a-judge task from existing solutions about critics. And the prompt enumerates failure modes such as data leakage, metric mismatch, schema errors, and unsupported claims, and constrains the model to one verdict token. We set $\Pi(\widehat{v}) =$ `No_Issues` only when $\widehat{v}$ contains `<No_Issues>` but not `<Issues_Found>`, and `Issues_Found` otherwise, since according to [18] and [42], LLM self-assessments are more trustworthy as risk flags than as clean verdicts.

Answer Signature Normalization. For K-way aggregation, raw comparison is impractical since two correct runs verbalise the same answer differently. We define a signature σ on $a = \text{Final}(\tau)$ via the priority chain

$$\sigma(a) = \begin{cases} \phi_1(a), & \text{if } a \text{ contains } \texttt{@name[value]} \text{ tokens,} \\ \phi_2(a), & \text{else if metric-value pairs are detectable,} \\ \phi_3(a), & \text{else if numeric or categorical labels are present,} \\ \phi_4(a), & \text{otherwise, with normalised text as fallback.} \end{cases}$$

where ϕ_1 extracts the InfiAgent-Bench-format `@name[value]` pairs [17], ϕ_2 for (metric, value) pairs, ϕ_3 for numeric or short categorical literals, and ϕ_4 applies whitespace and punctuation normalization. The chain runs from highest specificity to lowest, so trajectories agreeing at the formatted level always collide in signature space.

Signature-Based Majority Vote. Given samples $\{\tau_1, \ldots, \tau_K\}$ at temperatures $t_1 \le \cdots \le t_K$ with signatures $s_i = \sigma(\text{Final}(\tau_i))$, we group them by the resulting signature and return the earliest trajectory of the largest group $G_{s^\star}$, which will also be the lowest-temperature one under our assumption. It should also be noted that if the numbers of elements in all the groups get tied, a signature Tied will be returned and the result falls back to τ_1 with lowest temperature.

Review-Aware Aggregation. The combination of review and majority vote module, V_R, gates the vote by the q_review verdict. It first selects the clean subset C of trajectories flagged `No_Issues`, then runs majority vote over C when nonempty and over the full set otherwise.

When C is nonempty, voting only inside it filters out trajectories that the reviewer marked as suspicious before they can be counted. This is useful when the model is confidently wrong, as in plain majority voting, multiple bad runs can agree on the same wrong answer and outvote a correct one, but R tends to flag such runs and exclude them. When C is empty, the reviewer is not telling us anything useful, so we just run majority vote on all K trajectories instead of giving up, which still helps cut down sampling noise. In practice, C is small because the judgement from Π is conservative, with $|C|/K \approx 4\%$ on InfiAgent-Bench, so on most questions V_R ends up behaving exactly like plain majority vote V.

Algorithm 1 ReviewAwareAggregate

function ReviewAwareAggregate($T = [\tau_1, \ldots, \tau_K]$)
  $C \leftarrow \{ i \mid R(\tau_i) = \texttt{No\_Issues} \}$
  if $C \neq \emptyset$ **then**
    $\tau^\star \leftarrow$ MajorityVote($\{\tau_i \mid i \in C\}$)
  else
    $\tau^\star \leftarrow$ MajorityVote(T)
  end if
  if $\tau^\star$ = Tied **then**
    $\tau^\star \leftarrow \tau_1$
  end if
  return $\tau^\star$
end function

Runtime Protection. Running K trajectories per question over a 257-question benchmark amplifies resource leaks since each trajectory holds a Jupyter kernel inside a Docker container. DatawiseAgent's transition-count guardrails such as `max_planning_number` bound internal loops but neither cap external duration nor guarantee resource teardown. Therefore, we add two mechanisms to make the pipeline more practical. The first one sets a default budget $T_{\max} = 600$ seconds and enforces duration(τ_i) $\leq T_{\max}$ via `httpx.post`, discarding any timed-out τ_i so resumed runs are idempotent over completed IDs. The second one wraps each trajectory in a `try/finally` block that always issues a stop request to a new backend endpoint, which tears down the kernel and container through their `.stop()` methods with `auto_remove=True`. Together these bound the worst-case footprint at K concurrent sessions of duration $\leq T_{\max}$. In summary, this runtime protection assists us with executing the heavy majority voting pipeline.

5 EXPERIMENTAL EVALUATION

This section describes our experiment setup (§5.1), and evaluations of the confidence module (§5.2), memory module (§5.3), and review module (§5.4).

5.1 Experiment Setup

This section describes our experiment setup. We evaluate AI/DS-Agent on the widely-used Infiagent-DABench [17]. We compare AI/DS-Agent, and various ablated versions of AI/DS-Agent, with a state-of-the-art notebook-based AI/DS framework, DatawiseAgent [51].

Evaluation Metrics. For each method, we report Accuracy-by-Question (ABQ), Proportional Accuracy-by-Subquestion (PASQ), and Uniform Accuracy-by-Subquestion (UASQ). These kinds of metrics can evaluate the performance on the dataset more comprehensively.

LLM Agent. Due to the resource constriant of close-source LLM API's, we utilize GPT-4o-mini for compelete dataset evaluation and use GPT-5.4 for the subset with 7 easy questions, 7 normal questions and 6 hard questions.

Review Module Implementation. For the most resource-intensive review module, we implement the review state on top of the original DatawiseAgent FST by extending the agent configuration with a single `review.enabled` flag, which gates whether q_{review} is inserted before the idle transition. The review prompt and the conservative parser Π are added under the prompts and agents modules, and the verdict is persisted as a `ReviewNode` appended to the notebook so the aggregator can read it without re-invoking the LLM. The aggregation logic lives in a shared module reused by both plain majority vote and review-aware vote, and the trajectory loop in the evaluation runner dispatches on a `–method` flag among `baseline`, `q_review`, `mv`, and `rav`. Each trajectory is sampled at a configurable temperature, and we use $\{0, 0.7, 1.0\}$ throughout, with τ_1 at $t = 0$ so the tiebreak fallback to τ_1 also selects the lowest-variance trajectory. To avoid repeated review-stage LLM calls, V_R supports a seed-results option that reuses the verdicts from a prior q_review run as the first of the three trajectories. The runtime protections described in Section 3 are implemented as an `httpx` client-side timeout and a new backend stop endpoint that tears down the Jupyter kernel and Docker container from a client-side `try/finally` block.

Table 2: Confidence module ablation on the InfiAgent-Bench subset. Higher confidence score thresholds induces the agent to perform state inspection more often and leads to higher question accuracy.

Threshold	Q-Acc	SubQ-Acc	Avg. self-inpsection rounds per cell
0.75	53.88%	60.51%	0.29
0.9	55.95%	61.54%	0.43
0.95	**59.06%**	**64.04%**	**0.91**

5.2 Confidence Module: Reducing Cell Execution Errors

This section studies the effect of the confidence module on question accuracy; we run the full InfiAgent-DABench using various confidence thresholds and report relevant metrics in table 2.

As expected, our results suggest that setting a higher confidence threshold induces the agent to perform more on average more self-inspection rounds prior to generating each cell execution. This is effective in increasing the question accuracy, showcased in a 5.18% increase when increasing the threshold from 0.75 to 0.95. Notably, these self-inspection rounds allow the agent to avoid errors such as accessing non-existant columns in dataframes (fig. 2) which were commonly reported in prior work [51]. Furthermore, as hypothesized, this matches the effect of self-insepction in NL2SQL [6], where intermediate feedback has shown effectiveness in reducing accessing non-existing fields during SQL generation.

5.3 Memory Ablation: Diagnosing the Bottleneck

We further conducted a focused memory ablation on a 20-question subset of InfiAgent-Bench to understand whether the main bottleneck for the small-model setting comes from the memory structure itself. We compared four variants: the original DatawiseAgent workflow; `M1`, which enables debug-context trimming and successful-output truncation; `M2`, which additionally injects variable snapshots and summaries of prior failed debugging attempts; and `M3`, which keeps the memory design simple but adds a lightweight final-answer extraction safety net. The safety net does not recompute

Table 3: Memory ablation on a 20-question InfiAgent-Bench subset. Q-Acc is question-level accuracy, SubQ-Acc counts all subquestions uniformly, and Prop-Acc averages each question's fraction of correct subanswers.

Variant	Q-Acc	SubQ-Acc	Prop-Acc	Correct SubQ
Baseline	75.00%	79.41%	81.67%	27/34
M1	50.00%	61.76%	53.33%	21/34
M2	70.00%	76.47%	76.67%	26/34
M3	75.00%	**85.29%**	81.67%	**29/34**

the answer; it only tries to ensure that the final response contains the required `@name[value]` entries.

The results in Table 3 suggest that simply adding more memory is not the right fix. The most aggressive memory compression, `M1`, hurts performance substantially, dropping Q-Acc from 75.00% to 50.00%. Manual inspection shows that several of these failures are not computational failures: for example, on question 506 the agent computed the correct mean review count, 1013.53, but its final answer omitted the required `@average_reviews[1013.53]` format. Since the official evaluator extracts answers from `@name[value]` patterns, the response was marked incorrect despite containing the right number. This indicates that trimming context can remove or weaken the final formatting requirement for the small model, and that the resulting failure is better characterized as instruction-following or final-answer formatting rather than missing computational memory.

`M2` recovers much of the lost performance, but it still underperforms the baseline. This suggests that variable snapshots and failed-debug summaries can help the agent recover some execution state, but they do not directly solve the dominant failure mode. In some cases, the additional state also appears to introduce noise. For instance, on question 690, the required standard deviation after outlier removal matches the default sample standard deviation from `pandas.std()` (1.15). Some runs instead output 1.12, consistent with a population-standard-deviation interpretation. A variable snapshot can preserve the existence and value of variables, but it cannot force the model to choose the correct statistical convention; this is a question-understanding issue rather than a memory-structure issue.

The best variant is `M3`, which improves SubQ-Acc from 27/34 to 29/34 while leaving Q-Acc unchanged at 75.00%. This pattern is important: the improvement is real but narrow, mostly occurring at the subanswer and final-formatting level rather than at the full-question level. The clearest lesson is that final-answer verification is more valuable than more complex memory injection for this setting. The ablation also exposes limitations of the current evaluator. On question 450, `M3` produced the correct monthly wind-speed dictionary values, but the string differed from the label only in whitespace after colons, e.g., `{month_1:7.17,...}` versus `{month_1: 7.17,...}`. Because non-numeric structured answers are matched as raw strings, this was counted as wrong even though the semantic answer was correct.

Overall, the memory ablation changes our interpretation of the system bottleneck. For small models, the main challenge is not only what execution state is stored, but whether the model can reliably understand the question requirements and produce an answer that matches the required schema. Future improvements should therefore prioritize an answer verifier and canonical formatter: a verifier should check whether the final answer matches the question's required fields and statistical constraints, and a formatter should canonicalize numbers, lists, dictionaries, and categorical strings before evaluation. The memory module should remain lightweight, preserving necessary context and error evidence without adding excessive state that may distract the model.

Table 4: Results on the InfiAgent-Bench development set with GPT-4o-mini.

Method	ABQ	PASQ	UASQ	Correct
Baseline	71.98%	74.78%	77.43%	185/257
q_review	73.15%	76.10%	78.11%	188/257
Majority vote	73.15%	76.10%	77.79%	188/257
Review-aware vote	**73.93%**	**76.32%**	**78.31%**	**190/257**

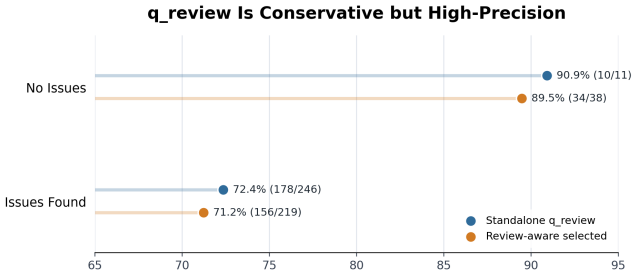

Figure 3: Accuracy Comparison of No_Issues and Issues_Found Trajectories

5.4 Performance of Review & Majority Vote Mechanism.

We evaluate our review & majority vote pipeline on the InfiAgent-Bench development set's 257 closed-form questions with GPT-4o-mini as the agent backbone. And the responses from LLMs are reformatted using the official `reformat.py` script before scoring with `eval_closed_form.py`.

Table 4 illustrates the performance of our review & majority vote module. Review-aware vote achieves the best result across all three metrics, with a 1.95-point ABQ improvement over baseline corresponding to 5 additional correct questions. Plain majority vote and q_review each contribute 3 questions individually, and review-aware vote adds 2 more on top of majority vote. In the full q_review run, Π assigned `No_Issues` to only 11 of 257 trajectories, so C is empty on the majority of inputs and V_R behaves like V outside that small slice.

Figure 3 also illustrates the effectiveness of our `q_review` module. Trajectories marked "No_Issues" achieve much higher accuracy than those marked "Issues_Found" in both standalone q_review and review-aware selection. In each setting, "No_Issues" trajectories reach 90.9% accuracy compared with 72.4% for flagged trajectories, while in review-aware voting, the gap remains similarly large with

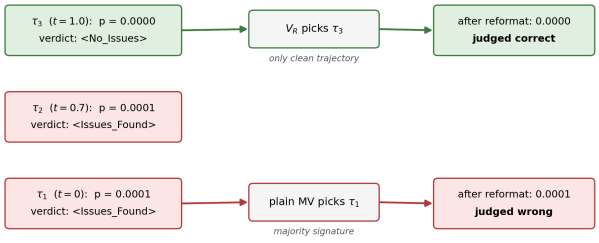

Figure 4: Question id 413, the one question in our run where V_R and plain majority vote select different trajectories. All three trajectories reach the same underlying computation but verbalise the p-value at different rounding precisions. The reviewer admits only τ_3, while plain majority vote follows the (τ_1, τ_2) signature majority. After the official `reformat.py` step, only τ_3 survives `eval_closed_form.py`.

89.5% versus 71.2%. This suggests that the review module is informative for selecting more reliable trajectories. At the same time, q_review is conservative, as it rarely assigns "No_Issues", but when it does, the clean verdict has high precision.

To illustrate a more specific sample, Question id 413 in Infiagent-Bench asks whether `Pclass` and `Fare` are correlated among Cherbourg passengers in the Titanic dataset, with reference p-value 0.0000 at four-decimal precision. Figure 4 traces the three trajectories. All three reach the same correct underlying computation, but τ_1 and τ_2 round the p-value as 0.0001 in their final answer block while τ_3 writes it as 0.0000. Because the reviewer flags the first two and admits only τ_3, the clean subset is the singleton $C = \{3\}$ and V_R returns τ_3. Plain majority vote sees a two-vs-one signature split and returns τ_1 as the earliest member of the majority group. This means that the gating thus carries some discriminative signal in our setting, but as currently parsed it functions closer to a precision-oriented format filter than to an audit of reasoning, since the results calculated by another two trajectories differ mainly because of the reformat process.

6 CONCLUSION

In this paper, we have proposed AI/DS-Agent, a new notebook-based AI and data science agent framework. It addresses shortcomings in existing notebook-based agent frameworks by introducing three modules that mimic well-studied, beneficial behavior of human practicioners: the confidence module which prompts the agent to inspect state data, the memory module for explicitly tracking and avoiding past failures, and the review module for reliably generating high-quality cell code via majority voting. We have shown that our system achieves accuracy improvements over a state-of-the-art implementation, DatawiseAgent on the widely-used Infiagent-DABench dataset.

REFERENCES

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 291–300.

[2] Muhammad Imam Luthfi Balaka, David Alexander, Qiming Wang, Yue Gong, Adila Krisnadhi, and Raul Castro Fernandez. 2025. Pneuma: Leveraging llms for tabular data representation and retrieval in an end-to-end system. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[3] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *Journal of machine learning research* 13, 2 (2012).

[4] Souti Chattopadhyay, Ishita Prasad, Austin Z Henley, Anita Sarma, and Titus Barik. 2020. What's wrong with computational notebooks? Pain points, needs, and design opportunities. In *Proceedings of the 2020 CHI conference on human factors in computing systems*. 1–12.

[5] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. Codet: Code generation with generated tests. *arXiv preprint arXiv:2207.10397* (2022).

[6] Kaiwen Chen, Yueting Chen, Nick Koudas, and Xiaohui Yu. 2025. Reliable text-to-sql with adaptive abstention. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–30.

[7] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. 2023. Universal self-consistency for large language model generation. *arXiv preprint arXiv:2311.17311* (2023).

[8] Yibin Chen, Yifu Yuan, Zeyu Zhang, Yan Zheng, Jinyi Liu, Fei Ni, Jianye Hao, Hangyu Mao, and Fuzheng Zhang. 2025. SheetAgent: Towards a Generalist Agent for Spreadsheet Reasoning and Manipulation via Large Language Models. In *Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) *(WWW '25)*. Association for Computing Machinery, New York, NY, USA, 158–177. https://doi.org/10.1145/3696410.3714962

[9] Andrew Crotty, Alex Galakatos, Emanuel Zgraggen, Carsten Binnig, and Tim Kraska. 2015. Vizdom: interactive analytics through pen and touch. *Proceedings of the VLDB Endowment* 8, 12 (2015), 2024–2027.

[10] Cody Dunne, Nathalie Henry Riche, Bongshin Lee, Ronald Metoyer, and George Robertson. 2012. GraphTrail: Analyzing large multivariate, heterogeneous networks while supporting exploration history. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 1663–1672.

[11] Ryan Ehrlich, Bradley Brown, Jordan Juravsky, Ronald Clark, Christopher R'e, and Azalia Mirhoseini. 2025. Codemonkeys: Scaling test-time compute for software engineering. *arXiv preprint arXiv:2501.14723* (2025).

[12] Philipp Eichmann, Emanuel Zgraggen, Carsten Binnig, and Tim Kraska. 2020. Idebench: A benchmark for interactive data exploration. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1555–1569.

[13] Hanxi Fang, Supawit Chockchowwat, Hari Sundaram, and Yongjoo Park. 2025. Large-scale Evaluation of Notebook Checkpointing with AI Agents. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–8.

[14] Xiang Fei, Xiawu Zheng, and Hao Feng. 2025. Mcp-zero: Proactive toolchain construction for llm agents from scratch. *arXiv e-prints* (2025), arXiv–2506.

[15] Jupyter Foundation. 2023. Jupyter MCP Server. https://github.com/datalayer/jupyter-mcp-server.

[16] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. 2024. Critic: Large language models can self-correct with tool-interactive critiquing, 2024. *URL https://arxiv. org/abs/2305.11738* (2024).

[17] Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, et al. 2024. Infiagent-dabench: Evaluating agents on data analysis tasks. *arXiv preprint arXiv:2401.05507* (2024).

[18] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2023. Large language models cannot self-correct reasoning yet, 2024. *URL https://arxiv. org/abs/2310.01798* 2 (2023).

[19] Jeremiah W Johnson. 2020. Benefits and pitfalls of jupyter notebooks in the classroom. In *Proceedings of the 21st Annual Conference on Information Technology Education*. 32–37.

[20] Project Jupyter. 2023. Jupyter Notebook. https://jupyter.org/.

[21] Mary Beth Kery, Bonnie E John, Patrick O'Flaherty, Amber Horvath, and Brad A Myers. 2019. Towards effective foraging by data scientists to find past analysis choices. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*. 1–13.

[22] Stepan Kulibaba, Artem Dzhalilov, Roman Pakhomov, Oleg Svidchenko, Alexander Gasnikov, and Aleksei Shpilman. 2025. Kompeteai: Accelerated autonomous multi-agent system for end-to-end pipeline generation for machine learning problems. *arXiv preprint arXiv:2508.10177* (2025).

[23] Doris Jung-Lin Lee, Dixin Tang, Kunal Agarwal, Thyne Boonmark, Caitlyn Chen, Jake Kang, Ujjaini Mukhopadhyay, Jerry Song, Micah Yong, Marti A Hearst, et al. 2021. Lux: always-on visualization recommendations for exploratory dataframe workflows. *arXiv preprint arXiv:2105.00121* (2021).

[24] Xingjun Li, Yizhi Zhang, Justin Leung, Chengnian Sun, and Jian Zhao. 2023. Edassistant: Supporting exploratory data analysis in computational notebooks with in situ code search and recommendation. *ACM Transactions on Interactive*

Intelligent Systems 13, 1 (2023), 1–27.

[25] Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, et al. 2024. Autokaggle: A multi-agent framework for autonomous data science competitions. *arXiv preprint arXiv:2410.20424* (2024).

[26] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* 12 (2024), 157–173.

[27] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Siheng Chen, et al. 2025. Ml-master: Towards ai-for-ai via integration of exploration and reasoning. *arXiv preprint arXiv:2506.16499* (2025).

[28] Stephen Macke, Hongpu Gong, Doris Jung-Lin Lee, Andrew Head, Doris Xin, and Aditya Parameswaran. 2020. Fine-grained lineage for safer notebook interactions. *arXiv preprint arXiv:2012.06981* (2020).

[29] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.

[30] Ahmed Masry, Xuan Long Do, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the association for computational linguistics: ACL 2022*. 2263–2279.

[31] Michael J Mior. 2024. Large language models for json schema discovery. *arXiv preprint arXiv:2407.03286* (2024).

[32] Fan Nie, Junlin Wang, Harper Hua, Federico Bianchi, Yongchan Kwon, Zhenting Qi, Owen Queen, Shang Zhu, and James Zou. 2026. DSGym: A Holistic Framework for Evaluating and Training Data Science Agents. *arXiv preprint arXiv:2601.16344* (2026).

[33] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).

[34] Jim Ormond. 2018. ACM Recognizes Innovators Who Have Shaped the Digital Revolution.

[35] Jeffrey M Perkel. 2018. Why Jupyter is data scientists' computational notebook of choice. *Nature* 563, 7732 (2018), 145–147.

[36] PBC Posit Software, PBC formerly RStudio. 2023. Posit RStudio. https://posit.co/.

[37] Maohao Ran, Zhenglin Wan, Cooper Lin, Yanting Zhang, Hongyu Xin, Hongwei Fan, Yibo Xu, Beier Luo, Yaxin Zhou, Wangbo Zhao, et al. 2026. CaveAgent: Transforming LLMs into stateful runtime operators. *arXiv preprint arXiv:2601.01569* (2026).

[38] Jon Saad-Falcon, E Kelly Buchanan, Mayee F Chen, Tzu-Heng Huang, Brendan McLaughlin, Tanvir Bhathal, Shang Zhu, Ben Athiwaratkun, Frederic Sala, Scott Linderman, et al. 2025. Shrinking the generation-verification gap with weak verifiers. *arXiv preprint arXiv:2506.18203* (2025).

[39] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[40] Arumoy Shome, Luis Cruz, Diomidis Spinellis, and Arie van Deursen. 2024. Understanding Feedback Mechanisms in Machine Learning Jupyter Notebooks. *arXiv preprint arXiv:2408.00153* (2024).

[41] Md Saeed Siddik, Hao Li, and Cor-Paul Bezemer. 2025. A systematic literature review of software engineering research on Jupyter Notebook. *Journal of Systems and Software* (2025), 112758.

[42] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2025. On the self-verification limitations of large language models on reasoning and planning tasks. In *International Conference on Learning Representations*, Vol. 2025. 98190–98243.

[43] The IPython Development Team. 2023. IPython Interactive Computing. https://ipython.org/.

[44] Eric-Jan Wagenmakers and Simon Farrell. 2004. AIC model selection using Akaike weights. *Psychonomic bulletin & review* 11, 1 (2004), 192–196.

[45] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).

[46] Mingyuan Wu, Meitang Li, Jingcheng Yang, Jize Jiang, Kaizhuo Yan, Zhaoheng Li, Hanchao Yu, Minjia Zhang, and Klara Nahrstedt. 2025. Aha Moment Revisited: Are VLMs Truly Capable of Self Verification in Inference-time Scaling? *arXiv preprint arXiv:2506.17417* (2025).

[47] Mingyuan Wu, Jingcheng Yang, Jize Jiang, Meitang Li, Kaizhuo Yan, Hanchao Yu, Minjia Zhang, Chengxiang Zhai, and Klara Nahrstedt. 2025. Vtool-r1: Vlms learn to think with images via reinforcement learning on multimodal tool use. *arXiv preprint arXiv:2505.19255* (2025).

[48] Doris Xin, Litian Ma, Shuchen Song, and Aditya Parameswaran. 2018. How developers iterate on machine learning workflows. In *IDEA Workshop at KDD*.

[49] Cong Yan and Yeye He. 2020. Auto-suggest: Learning-to-recommend data preparation steps using data science notebooks. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1539–1554.

[50] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems* 37 (2024), 50528–50652.

[51] Ziming You, Yumiao Zhang, Dexuan Xu, Yiwei Lou, Yandong Yan, Wei Wang, Huamin Zhang, and Yu Huang. 2025. DatawiseAgent: A Notebook-Centric LLM Agent Framework for Adaptive and Robust Data Science Automation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 1099–1123.

[52] Emanuel Zgraggen, Robert Zeleznik, and Steven M Drucker. 2014. Panoramic-Data: Data analysis through pen & touch. *IEEE transactions on visualization and computer graphics* 20, 12 (2014), 2112–2121.

[53] Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. ToolQA: A Dataset for LLM Question Answering with External Tools. *arXiv preprint arXiv:2306.13304* (2023).